Contents

1 Introduction to Explainable AI **2**
1.1 Taxonomy . . . 2
1.2 Global vs Local interpretability . . . 2
1.3 Making explanations . . . 2

2 Linear Models **2**
2.1 Linear Regression . . . 2
2.2 Lasso - Least Absolute Shrinkage and Selection Operator . . . 2
2.3 Logistic Regression . . . 2
2.4 Linear Regression Extensions . . . 3
2.4.1 Generalized Linear Models . . . 3
2.4.2 Feature Interactions . . . 3
2.4.3 Generalized Additive Models . . . 3

3 Tree Models **4**
3.1 Decision Trees by CART algorithm . . . 4
3.2 Decision Rules . . . 4
3.3 RulerFit . . . 4

4 Mode Agnostic and Example based **5**
4.1 Model Agnostic . . . 5
4.2 Examples Based . . . 5

5 Local Model Agnostic **5**
5.1 ICE - Individual Conditional Expectation . . . 5
5.2 LIME - Local Interpretable Model-Agnostic Explanations . . . 5
5.3 Shapley values & SHAP . . . 6
5.3.1 Shapley values . . . 6
5.3.2 SHAP - SHapley Additive exPlanations . . . 7

1 Introduction to Explainable AI

1.1 Taxonomy

By Interpretability Type

- Intrinsic Interpretability
- Post-hoc Interpretability

By Result Type

- Feature summary statistics
- Feature summary visualizations
- Model internals (weights, tree structure)
- Data Point base (counterfactuals, prototypes)
- Intrinsic surrogates

1.2 Global vs Local interpretability

Global Interpretability How well the trainied model makes predictions overall. Which features are important for predictions overall. Feature interactions.

Local Interpretability Why the model made a particular prediction for one instance Locally a complex model may behave in a simple way.

1.3 Making explanations

When making explanations, we need to keep the human factor in mind. They should be

- Contrastive: Provide "why this, not that?" with a clear reference point.
- Selected: Limit to the top 1–3 drivers.
- Social: Match audience knowledge and context.
- Abnormality: Highlight unusual but influential factors.
- Fidelity: Stay faithful to model behavior (and reality when possible).
- Beliefs: Acknowledge/prioritize known priors or add constraints if needed.
- Generality: Indicate coverage/support when relevant.

2 Linear Models

Using inherently interpretable algorithms.

2.1 Linear Regression

Predicts target as weighted sum of inputs.

$$y = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p + \epsilon \tag{1}$$

Estimated usually by Ordinary Least Squares (OLS)

$$\hat{\beta} = \underset{\beta_0,\ldots,\beta_p}{\arg\min} \sum_{i=1}^{n} \left(y^{(i)} - \left(\beta_0 + \sum_{j=1}^{p} \beta_j x_j^{(i)} \right) \right)^2 \tag{2}$$

Each weight represents the effect of one feature, holding others constant. We can compute **confidence intervals** for each coefficient.

Table 1: Regression Output

Feature	Weight$\hat{\beta}_j$	SE	$\lvert t \rvert$
Temperature	110.7	7.0	15.7
Humidity	-17.4	3.2	5.5

Regression Output

- Weight: Estimated influence of x_i over the outcome y
- SE (Standard Error): Uncertainty (variance) of the estimated coefficient
- $|t|$ t-statistic: Strength of evidence that $\beta_j \neq 0$

$$t_{\beta_j} = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)} \tag{3}$$

Large, certain weights = important features. Small or uncertain weights = weak evidence. Features wit highest $|t|$ values are most significant.

- Effect Plot: $effect_j^i = \beta_j \cdot x_j^i$ Contribution of feature j to prediction for instance i.
- Different encodings for variables.

Notes on Linear Regression

- Perfectly understood
- Only captures linear relationships
- Ignores feature interactions, must be added manually ($x_1 x_2, x$ etc.)
- Lower accuracy compared to more flexible Models.

2.2 Lasso - Least Absolute Shrinkage and Selection Operator

Using models that keep only the relevant features helps with interpretability.

- Adds L1 Norm, Performs feature selection, regularization
- Enforces sparsity in Linear model

$$\underset{\beta}{min} \frac{1}{n} \sum_{i=1}^{n} \left(y^{(i)} - x_i^T \beta \right)^2 + \lambda \left\| \beta \right\|_1 \tag{4}$$

As λ gets increased, only the most important features remain. This increases interpretability.

2.3 Logistic Regression

- Classification problems with two classes.
- Models Probability of one class given the features.
- Extension of linear regression for classification.
- Uses the output of a linear regression and maps it to a Probability.

Logistic / Sigmoid function

$$logistic\,(\eta) = \frac{1}{1 + \exp -\eta} \tag{5}$$

Logistic Model

$$P\left(y^{(i)}=1\right)=\frac{1}{1+\exp(-(\beta_0+\beta_1 x_1^{(i)}+\cdots+\beta_p x_p^{(i)}))} \quad (6)$$

- Linear predictor is transformed via the logistic function.
- Coefficients do not act linearly on the probability. Interpretation is happens on the log-odds scale.
- A one unit increase in x_j multiplies the odds of $y=1$ vs. $y=0$ by a factor of $\exp(\beta_j)$
- Predicts probabilities, which allows for better decision making. Pred with 99% probability is more confident than one with 51%.
- Relationships between features and outcomes remain transparent.

2.4 Linear Regression Extensions

Linear Regression Models can be extended to handle some things that normal linear Regression can not.

- y distribution is not Gaussian: Use Generalized Linear Models (GLMs)
- The features interact: Add interactions manually
- The relationship between x and y is not linear: Use Generalized Additive Models (GAMs)

2.4.1 Generalized Linear Models

Use this if the output y follows a Non-Gaussian distribution. For example if we want to predict counts. They follow a Poisson distribution (discrete values, $Var(y)=\mathbb{E}[y]$)

$$g(\mathbb{E}[Y|x])=\beta_0+\beta_1+\cdots+\beta_p x_p=x^T\beta \quad (7)$$

It has three components

- **Random Component**: A distribution of the exponential family that defines $\mathbb{E}_Y$
- **Systematic component**: Linear predictor $\eta=X^T\beta$. (LR Term)
- **Link function**: Function $g(\cdot)$ that links the mean to η

Our models prediction is: $\mathbb{E}[y]=g^{-1}(x^T\beta)$ which could be 7.6 for example customer count. But with GLM, since we have defined the distribution of y (Poisson), we can make several inferences using the predicted mean. Of course this means, that we have to choose $\mathbb{E}[y]$ correctly, according to our data, or the inferences will be wrong.

- Probability that we get more than 15 customers, when 7.6 was predicted as mean.
- Catch outliers
- Calculate the variance of our prediction.

Distributions and their canonical link functions

- Poisson: Log: $g(\mu)=\eta$
- Binomial: Logit: $\ln\left(\frac{\mu}{1-\mu}\right)=\eta$
- Normal: Identity: $\mu=\eta$

GLMs transform model outputs y

2.4.2 Feature Interactions

- Features interact with each other.
- E.g. Nr of bike Rentals: Temperature and Working/Non-Working day. on days of people only rent bikes when its warm. On working days, people ride to work regardless of weather.
- In LR we need to model these interactions explicitly. Add Interaction terms (Product of two features)
- Modelling of interactions between different feature types:
  - `Cat` x `Num`: Existence of category x Num feature value.
  - `Cat` x `Cat`: `AND` between one hot encoded features.
  - `Num` x `Num`: Multiply the features

2.4.3 Generalized Additive Models

In a linear model, the increase of a feature by 1 has the same effect everywhere (over the whole range of that features values) But there is not always a linear relationship over the entire range. (Growth, saturation, decline). We need to model these nonlinear relationships.

E.g. 10 to 11 degrees, bike rentals increase, but 40 to 41 degrees they decline (to hot).

Approaches to Non-Linearity

- Replace features with transformed versions. $log(x), \sqrt{x}, x$
- Discretizise categorical features into Bins/Quantiles (one hot encoded) $\rightarrow$ model becomes step function.
- GAMs: Let model learn smooth non-linear functions of features.

GAMs

$$g(\mathbb{E}_Y[y|x])=\beta_1+f_1(x_1)+f_2(x_2)+\cdots+f_p(x_p) \quad (8)$$

- GAMs are GLMs but the features are transformed non-linearly with a function.
- GAMs allow GLMs to learn nonlinear relationships.
- $f_i(x_i)$ is usually a learned spline function.
  - Think of them as k new features that are obtained via k spline functions. The spline functions outputs combined via a weighted sum model the non-linearity.
  - The range is split into k sections/knots (e.g. by percentile)
  - For each section, a spline is fit e.g cubic $y=ax^3+bx^2+cx+d$
  - Then the outputs of these splines is combined via a weighted sum.

Maintain interpretability, because the features are still additive across features.

GAMs transform model inputs x with learned non-linear functions.

3 Tree Models

- Recursively split dataset into subsets.
- Each split aims to make the subsets as homogenous as possible wrt. the target.
- Preds: Classification → Majority class in leaf node. Regression → Mean of target variable of instances in leaf node.
- Trees can model nonlinear and feature interactions naturally.

3.1 Decision Trees by CART algorithm

Classification and Regression Trees. Splits determined by Regression: variance of target y, Classification: Gini impurity/entropy.

1. Test all possible cut-off values for splits
2. Check variance/impurity reduction per split
3. Select threshold that achieves the best split
4. Repeat recursively for child nodes

Prediction

$$\hat{y} = \hat{f}(x) = \sum_{m=1}^{M} c_m \cdot \mathbb{1}\{x \in R_m\} \tag{9}$$

- c_m: Prediction in leafe node (Avg y or majority class over training instances in subset R_m)
- Indicator function: If x is in Region m

Only one element of the sum will be active.

Splitting Regression

$$Var\,(y) = \frac{1}{N}\sum_i (y_i - \bar{y})^2 \tag{10}$$

Splitting Classificaiton

$$G = \sum_k p_k(1 - p_k) \qquad \text{Gini Index} \tag{11}$$

$$H = \sum_k p_k \log(p_k) \qquad \text{Entropy} \tag{12}$$

Stop Criteria We stop splitting, when one or more conditions are met.

- Nr of samples in a node before splitting
- Nr of saples in a terminal node
- Max depth
- Not enough reduction in impurity or variance.

Final Tree partitions the input space.

Interpreting Decision Trees Follow the decision path from root to leaf. Splits along path are connected by logical `AND`

- **Global Feature importance:** Important features are involved in more splits. And important splits reduce the variance/impurity by a large amount.

$$\text{Feature importance } = \frac{\text{Total reduction by feature}}{\text{Sum of all reductions}} \cdot 100 \tag{13}$$

  Where Reduction by feature is the sum of all reductions at each split where the feature is used. Sum of all reductions is the reduction of all features combined.

- **Local:** To get the contribution of a feature to the prediction:

$$\hat{f}(x) = \bar{y} + \sum_{d=1}^{D} \text{split.Contrib}(d, x) = \bar{y} + \sum_{j=1}^{p} \text{feat.contrib}(j, x) \tag{14}$$

  Instance pred = mean outcome + feature contributions. Contribution could be change in prediction by choosing to take one branch (|That subsets mean - supergroups mean|).

Notes

- Shallow trees (depth ≤ 3) provide compact explanations.
- Small input changes can cause big jumps in output.
- Trees quickly become to difficult to interpret with increasing depth

3.2 Decision Rules

- `IF` condition `THEN` prediction.
- Rules build from simple statements, can be combined or single rules. Intuitively interpretable
- Support: percent of data where rules condition applies
- Accuracy: Percentage of instances where the prediction is correct of instances where the rule is applied.

Automatically produced by tree ml models

3.3 RulerFit

Combines simplicity of linear models with flexibility of decision trees. Learn a sparse linear model that includes original features and rules capturing nonlinear interactions(Lasso $L1$ norm added).

$$\hat{y} = \beta_0 + \sum_{j=1}^{p} \beta_j x_j + \sum_{k=1}^{K} \alpha_k r_k \tag{15}$$

Rule Generation

1. Fit an ensemble of trees on the data (m trees generates $K = \sum_{m=1}^{M} 2(t_m - 1)$ rules, where t_m is number of leaf nodes in tree m). Generate trees of different depths for diversity.
2. Get all unique paths from root to leaf, each one is a decision rule

3. Use rules as binary features.
4. Add them as binary features to the model $r = 1$ if rule is active $r = 0$

4 Mode Agnostic and Example based

4.1 Model Agnostic

- Compare multiple models based on model-independent metrics
- Potentially better predictive performance Helps us trust black-box models better.

4.2 Examples Based

- Explain models using specific instances.
- Only useful if instances are human interpretable
- Useful for Images/text, where features carry rich meaning
- Easily understandable explanations: This patient was similar to this other, that's why their prediction is similar too.

Decision trees, kNN use example based prediction

Types of examples

- **Prototypes**: Representative examples (similar)
- **Counterfactuals:** Minimal changes to change prediction. How much to change x_i so that the loan gets approved. Learn about model.
- **Adversarial:** Counterfactuals designed to fool the model. Exploit the models prediction. Malicious.

5 Local Model Agnostic

5.1 ICE - Individual Conditional Expectation

How a samples prediction changes, when all but one features are fixed and then the remaining feature gets replace by values from a grid

1. Keep all but one features fixed.
2. Replace remaining feature with values from a grid of reasonable values for that feature.
3. Record how the prediction changes while varying that feature.
4. Plot the change. With y axis, the predicted value, x axis the values of the feature.

- Shows how the instances prediction changes when a feature changes.
- Visualizes the dependence of the prediction on a feature for each instance separately

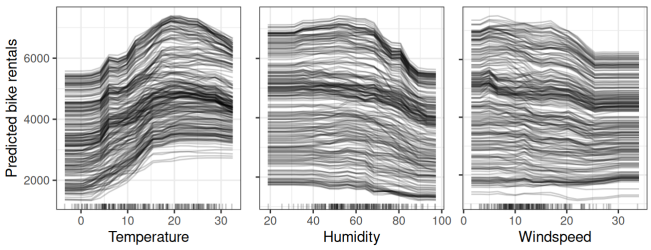

Figure 1: ICE-Plot

ICE-Plot

Centered ICE-Plot Since it is hard to see sometimes, if the individual curves are different from one another, they are usually centered. To center the curves subtract the intercept of each curve. They start at different intercepts, because the data points are different (duh).

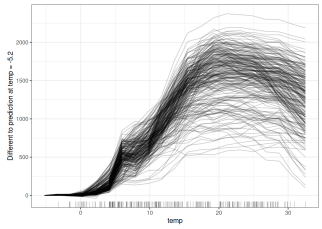

Figure 2: Centered-ICE Plot

Connected to ICE; A Partial Dependence Plots is just the average of all ICE curves. Often, when the ICE curves are too crowded,

Derivative ICE-Plot Instead of the raw data, plots the derivative of each ICE-curve.

- Show where and how strongly predictions change
- Derivatives tell in which direction change occurs.
- Makes it easy to spot in which ranges the predictions change.
- Spot non-linearity (linear relationship is flat line)

5.2 LIME - Local Interpretable Model-Agnostic Explanations

Explain model predictions by using local surrogate models.

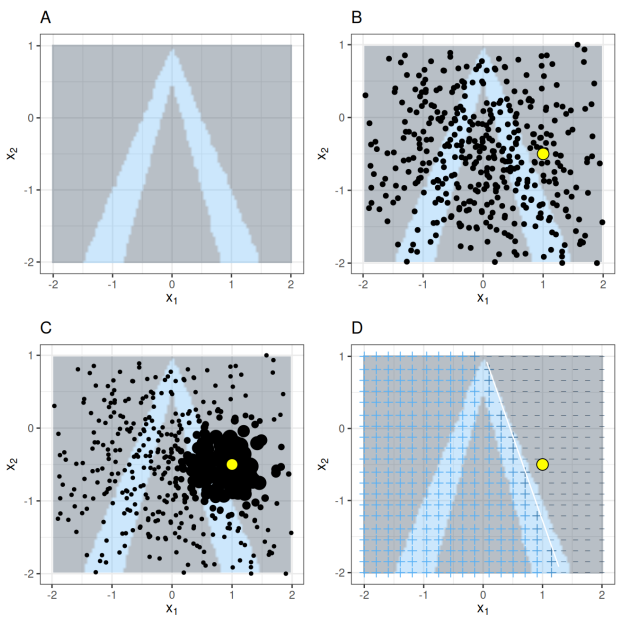

Figure 3: LIME explanations $a \to b \to c \to d$
Background is the black-box models predictions

LIME

$$\text{explanation}(x) = \arg\min_{g \in G} L\left(\hat{f}, g, \pi_x\right) + \Omega(g) \tag{16}$$

where

- explanation is the model g, that minimizes the Loss.
- Loss measures how close the explanation model g is to the original $hatf$.
- π_x: Defines how large the neighborhood around instance x is, that we consider for the explanations. This is done by weighting the points in the dataset using an exponential kernel.
- $\Omega(g)$: Complexity of the model. Has to be set by the user. LIME only optimizes the loss part.

LIME Algorithm

1. Choose instance to explain.
2. Generate a new dataset by perturbing the instance (generate variations of it to generate similar instances).
   - Tabular: Get μ, σ from the dataset (requires a few samples) and then swap the instances features with values drawn from the normal distribution. Maybe its also sampled straight from the datasets mass center.
   - Images: Replace super-pixels (groups of connected pixel with similar colors) with a different color.
   - Text: Mask out some words.
3. Make prediction for the perturbed dataset using the black-box model
4. Fit the interpretable model (e.g. Linear Regression) using Equation 16. Points close to the to be explained instance are weighted more strongly in the loss function (by the exponential kernel).

Notes

- Sensitive to kernel with choice.
- Neighborhood definition is pretty arbitrary.
- Sampling features ignores correlations

5.3 Shapley values & SHAP

5.3.1 Shapley values

Goal: Fairly attribute the difference between the average and a specific prediction to all input features.

In Theory

$$\phi_j(val) = \sum_{S \subseteq \{1,\dots,p\} \setminus \{j\}} \underbrace{\frac{|S|!\,(p-|S|-1)!}{p!}}_{\text{weight}} \underbrace{(val\,(S \cup \{j\}) - val(S))}_{\text{Marginal contribution of } j} \tag{17}$$

ϕ_j, the Shapley value is the average marginal contribution of feature j across all possible feature coalitions that j can join.

- S: Subset of features that does not include j
- $val_x(S)$: Prediction value using only features in S (from the instance that we want to explain.)
- $val_x(S \setminus j)$: Prediction value using features in S and j
- $\frac{|S|!(p-|S|-1)!}{p!}$: Probability that j can make that contribution. E.g. Number of ways, that j can join a coalition of three other features.

The $val_{\mathbf{x}}(S)$ is the predicted outcome of the model minus the average prediction using the defined features (S).

$$val_{\mathbf{x}}(S) = \int \hat{f}(x_1, \dots, x_p) d\mathbb{P}_{X_C} - \mathbb{E}[\hat{f}(\mathbf{X})] \tag{18}$$

This essentially tells us to integrate over the probability distribution of the features not in the coalition X_C and get the average prediction.

$$\sum_{j=1}^{p} \phi_j(\hat{f}) = \hat{f}(\mathbf{x}) - \mathbb{E}[\hat{f}(\mathbf{X})] \tag{19}$$

The sum of Shapley values fully explains the difference from the mean prediction.

In Practice

- Because in ML Models we need the same amount of input features for all predictions, we can't just leave them out.
- All possible sets of features have to be evaluated. This quickly becomes computationally intractable for more than a few features.

Approximation using Monte-Carlo sampling

$$\hat{\phi}_j = \frac{1}{M} \sum_{m=1}^{M} \left(\hat{f}(\mathbf{x}_{+j}^{(m)}) - \hat{f}(\mathbf{x}_{-j}^{(m)})\right) \tag{20}$$

1. Replace random features in x with features from a randomly sampled data point z

2. Make one prediction for x_{+j}, where j is taken from x and one with x_{-j} where j is from z.
   $\rightarrow$ Marginal contribution of j
3. Compare predictions
4. Repeat M times and take the mean difference as the Shapley value.

Notes

- Computationally expensive
- Requires access to the data distribution (a dataset)
- Only attribution method that satisfies following properties,which together can be considered a definition of a *fair payout*.
  - Efficiency: Contributions sum to prediction minus mean.
  - Symmetry: Features with identical effects get equal contributions.
  - Dummy: Features that don't change the prediction get contribution 0
  - Additivity: For combined models, Shapley values add up accordingly.
- Contrastive explanations: Instead of average could compare to a subset or single data point.
- Not Local in some sense. Positive value does not mean that increasing the feature would increase the prediction. Interpret wrt. Reference dataset

5.3.2 SHAP - SHapley Additive exPlanations

When we compute Shapley values, *we want to know how much each feature contributed on average.*

SHAP adjusts the Shapley value computation for the needs of ML models. Make it more efficient, and offer ways to approximate them.

Theory In SHAP we want to explain model predictions as a linear combination of feature contributions (Shapley values).

$$g(\mathbf{z}') = \phi_0 + \sum_{j=1}^{M} \phi_j z'_j \tag{21}$$

- g Explanation model
- $z' = (z'_1, \ldots, z'_M)^T \in \{0,1\}$ Coalition vector: 1=feature present, 0=feature absent.
- M Maximum coalition size
- $\phi_j \in \mathbb{R}$ Feature attribution for feature j

KernelSHAP Instead of exactly calculating the outputs for all 2^k possible coalition we fit a linear regression model. Where the input z' is the coalition vector which signals if a feature is present and its prediction target y is the prediction from the original model for an instance where the not included features are replaced by features from random instances.

Like this the coefficients are essentially, what each feature contributed to the prediction and ϕ_0 is the mean prediction, which is what Shapley values are. Or more precisely:

1. Sample coalition vectors $z'_k \in \{0,1\}^M, k \in \{1, \ldots, K\}$
2. Get prediction for each z'_k convert it to a feature vector and apply the model $\hat{f} : \hat{f}\left(h_x(z'_k)\right)$
3. Compute the weight for each coalition z'_k with the SHAP kernel $\pi_x(z'_k)$. How much we can learn from this coalition about the feature contributions.
4. Fit weighted linear regression model
5. Return Shapley values ϕ_k the coefficients from the linear model.

Where

- $h_x(z'_k) = z$ where $h_x(z'_k) : \{0,1\}^M \rightarrow \mathbb{R}^p$ maps 1s to the features from instance x, 0s to features from another random sample from the data.
- $\hat{f}$ Is the model whose predictions we want to explain.

The SHAP Kernel.

$$\pi_{\mathbf{x}}(\mathbf{z}') = \frac{(M-1)}{\binom{M}{|\mathbf{z}'|}|\mathbf{z}'|(M-|\mathbf{z}'|)} \tag{22}$$

- M Maximum coalition size
- $|z'|$ Number of present features in instance z'

Assigns a weight to each coalition. If a coalition consists of a single feature, we can learn about this feature's isolated main effect on the prediction. If a coalition consists of all but one feature, we can learn about this feature's total effect (main effect plus feature interactions).

To get the most value out of the K coalitions. We can sample more efficiently, by starting with the coalitions that will have higher weights and move to those with lower weights until we have K coalitions.

With this we can train the linear model with a weighted sum of squares.

$$L(\hat{f}, g, \pi_{\mathbf{x}}) = \sum_{\mathbf{z}' \in \mathbf{Z}} [\hat{f}(h_{\mathbf{x}}(\mathbf{z}')) - g(\mathbf{z}')]^2 \pi_{\mathbf{x}}(\mathbf{z}') \tag{23}$$

- Z Training data containing our coalition vectors z'

TreeSHAP Variation of SHAP for tree-based models. It is a faster model-specific alternative to KernelSHAP. Tree has only limited amount of different possible predictions, so many coalitions will have the same prediction and we make the same computations multiple times. E.g. for depth 5 there are 32 possible output.

We can exploit this and make each computation only one time and then we just need to weight those according to how many coalitions would have ended in that prediction.

- Features that do not influence then model can Still get non-zero SHAP values.
- Still much faster than KernelSHAP.

Visualizations SHAP Feature Importance

$$I_j = \frac{1}{n} \sum_{i=1}^{n} |\phi_j^{(i)}| \tag{24}$$

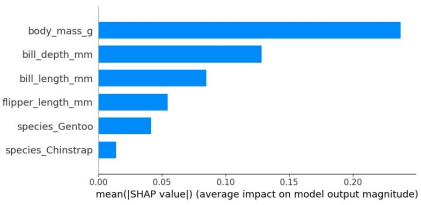

Figure 4: SHAP Feature Importance

SHAP Summary Plot

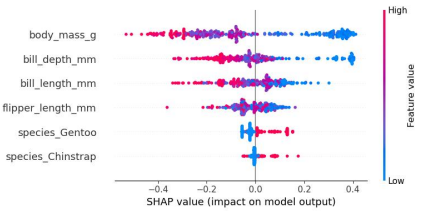

Figure 5: SHAP Summary

Overlapping points are offset in the y-axis direction, gives a sense of the distribution of Shapley values per features.

SHAP Dependence Plot

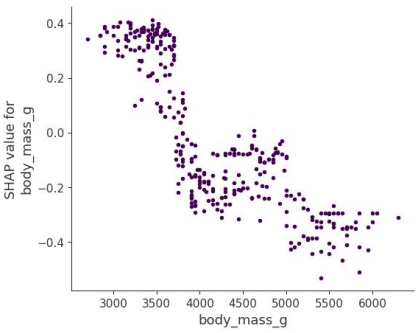

Figure 6: SHAP Dependence Plot

- Like PDP/ALE, but shows full distribution, not only average
- Variance reveals interactions. (interactions = higher variance)

We can plot the interaction values in the dependence plot.

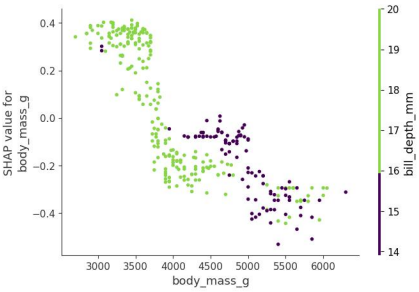

Figure 7: Plots the dependence plot for one feature and the color signals the interaction with another feature.

Clustering SHAP

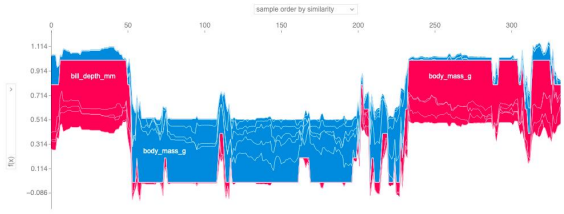

Figure 8: Stacked SHAP explanations clustered by explanation similarity

- Force Plots for all instances side by side, clustered by similarity
- Identify distinct risk profiles or behavior groups

SHAP Notes

- Shapley Advantages
- Rich sets of plots
- Bridges LIME and Shapley, same additivity as LIME but grounded in Axioms.
- KernelSHAP is slow
- unrealistic combinations of features when features are dependent (like all permutation based methods)
- TreeSHAP uses conditional expectations. Can attribute importance to non-influential correlated features.
- Shapley disadvantages.

Careful when features are strongly correlated or when you need fast, sparse explanations.